



Attribute Selection



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Attribute Subset Selection

Before selecting attributes, it is necessary to understand which types of attributes should be eliminated from the dataset.

Redundant Attributes

Duplicate information contained in one or more other attributes, adding no additional predictive value to the model.

Example: The purchase price of a product and the tax paid on that purchase are redundant, since the tax is directly calculated from the price.

The identification and removal of these attributes improves computational efficiency, reduces overfitting, and increases the interpretability of the resulting models.

With these concepts in mind, we can structure the principles that guide the selection process.

Irrelevant Attributes

Do not contain useful information for the data mining task at hand, only adding unnecessary noise and complexity.

Example: Student IDs are irrelevant for the task of predicting students' GPA, as they are merely arbitrary identifiers with no relation to academic performance.

Principles of Attribute Selection

Attribute selection follows a systematic architecture that combines different components to identify the optimal subset of features.

Subset Generation

Strategies to explore the space of possible attribute combinations

Evaluation

Metrics to quantify the quality of each candidate subset

Stopping Criterion

Conditions that determine when to terminate the search process

Validation

Verification of the selected subset's performance on independent data

Reference: Dash, M., Liu, H., (1997), "Feature Selection for Classification", Intell. Data Anal., v. 1, n. 3, p. 131–156.

These four components manifest in distinct ways in the methods we will explore next.

Feature Selection Methods

Five algorithms are covered in this presentation — each implementing the principles of generation, evaluation, stopping, and validation in its own way. All are available in the `daltoolbox` library through a unified interface: `instantiate`, `fit()` and `transform()`.

Information Gain

Ranks features by the reduction in entropy each provides to the categorical target.

`feature_selection_info_gain()`

Relief

Weights features by their ability to distinguish instances of different classes via nearest-neighbor comparisons.

`feature_selection_relief()`

Stepwise Search

Iteratively adds or removes predictors from a GLM guided by the AIC criterion.

`feature_selection_stepwise()`

LASSO

Applies L1 regularization to linear regression, shrinking irrelevant coefficients exactly to zero.

`feature_selection_lasso()`

FSS

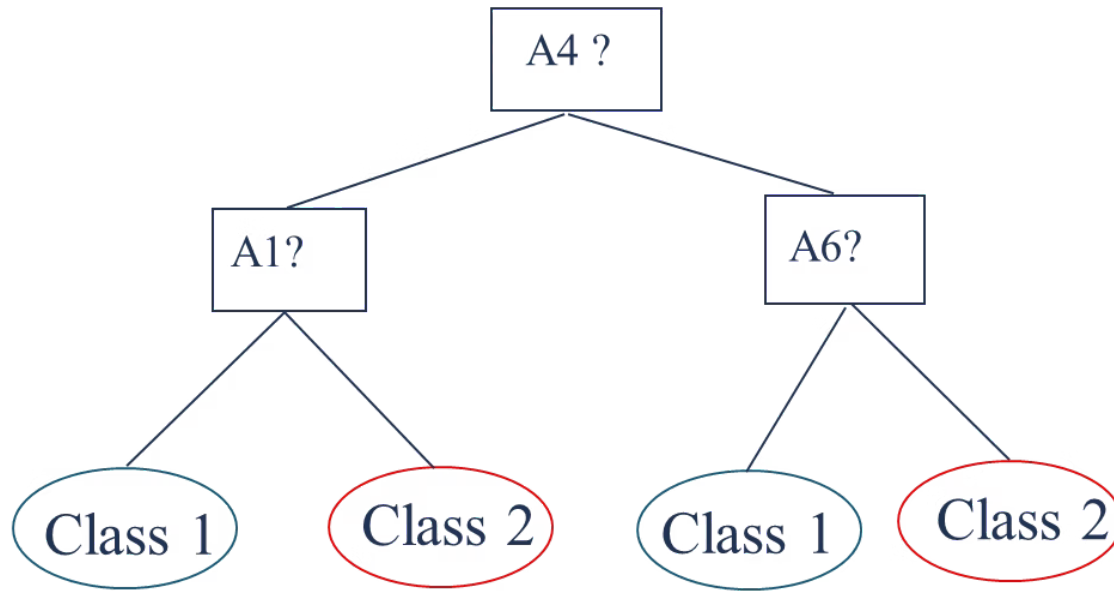
Forward stepwise subset search that maximizes adjusted R^2 using exhaustive search via the leaps package.

`feature_selection_fss()`

❏ **General principle:** All methods must be fitted (`fit()`) only on the training set to avoid data leakage. The target column must be explicitly specified at instantiation.

Information Gain

A filter-based ranking method that selects features by how much they reduce the entropy of the target class.



$$IG(A) = H(Y) - H(Y|A)$$

Calculate Initial Entropy

Compute $H(\text{target})$ before any split.

Evaluate Each Attribute

Compute $H(\text{target} | A_i)$ for each attribute.

Compute Information Gain

$IG(A_i) = H(\text{target}) - H(\text{target} | A_i)$.

Rank and Select

Retain the top-k attributes by IG score.

Next, the practical implementation of Information Gain using the daltoolbox library in R.

Information Gain

First method in practice: Information Gain ranks features by the reduction in entropy each one provides to the categorical target.

```
library(daltoolbox)

iris <- datasets::iris
iris$Species <- factor(iris$Species)

fs <- feature_selection_info_gain(
  "Species", top = 2
)
fs <- fit(fs, iris)

print(fs$selected)
## [1] "Sepal.Width" "Sepal.Length"
```

```
print(fs$ranking)
##   feature   score
## 1 Sepal.Width 0.3571324
## 2 Sepal.Length 0.0000000
## 3 Petal.Length 0.0000000
## 4 Petal.Width 0.0000000

iris_fs <- transform(fs, iris)
head(iris_fs)
##   Species Sepal.Width Sepal.Length
```

What it is

Measures the reduction in target entropy provided by each feature.

Parameter top

Defines how many features will be retained after ranking.

Result

`Sepal.Width` achieved a score of 0.357 — the most informative. The others scored zero.

Transformation

`transform()` returns the dataset reduced to the selected features.

📌 **Key concept:** Information Gain = $H(\text{target}) - H(\text{target} \mid \text{feature})$. The higher the value, the more the feature reduces uncertainty in the target. Ref.: Quinlan (1993).

Relief, presented next, uses a different strategy — it compares neighboring instances to estimate the relevance of each feature.

Relief

Relief is a filter-based feature selection method that estimates the relevance of each feature by analyzing the local structure of the data — specifically, how well each feature distinguishes between instances of the same and different classes.

Random Sampling

Randomly selects m instances from the training set (controlled by parameter m).

Find Nearest Hit

For each sampled instance, finds its nearest neighbor from the same class (nearest hit).

Find Nearest Miss

Finds the nearest neighbor from a different class (nearest miss).

Update Weights

Increases the weight of features that differ between the instance and the miss; decreases the weight of features that differ between the instance and the hit.

Rank Features

After m iterations, features are ranked by their accumulated weight.

Key Properties

- Filter method: independent of the learning algorithm
- Captures feature interactions via neighborhood structure
- Sensitive to the parameter m (number of sampled instances)
- Works well for classification with multiple classes (ReliefF variant)

Advantages & Limitations

- **Advantages:**
 - Detects feature interactions that univariate methods miss
 - Robust to noisy features
- **Limitations:**
 - Computationally heavier than Information Gain ($O(m \cdot n \cdot p)$)
 - Results depend on the random seed (use seed for reproducibility)

Next, the practical implementation of Relief using the daltoolbox library in R.

Reference: Kononenko, I. (1994). Estimating attributes: Analysis and extensions of RELIEF. In European Conference on Machine Learning (ECML), pp. 171–182. Springer.

Relief

While Information Gain evaluates features independently, Relief considers the local structure of the data — comparing each observation with its nearest neighbors from the same and different classes.

```
library(daltoolbox)
iris <- datasets::iris
iris$Species <- factor(iris$Species)

fs <- feature_selection_relief(
  "Species", top = 2, m = 50, seed = 1
)
fs <- fit(fs, iris)
```

```
print(fs$selected)
## [1] "Petal.Width" "Petal.Length"

print(fs$ranking)
##   feature   score
## 1 Petal.Width 0.2108333
## 2 Petal.Length 0.1633898
## 3 Sepal.Width 0.1508333
## 4 Sepal.Length 0.0600000

iris_fs <- transform(fs, iris)
head(iris_fs)
##   Species Petal.Width Petal.Length
```

What it is

Estimates relevance by comparing each observation with its nearest neighbor (hit) and farthest neighbor (miss).

Parameters

`m = 50` samples evaluated; `seed = 1` ensures reproducibility.

Result

Relief selected `Petal.Width` and `Petal.Length` — different from Information Gain.

When to use

Useful for classification problems with interactions between features.

📌 **Key concept:** Relief penalizes features that poorly separate neighboring classes. Higher scores indicate greater discriminative power. Ref.: Kononenko (1994).

Stepwise Search, presented next, shifts the paradigm: instead of ranking features individually, it iteratively builds a GLM model.

Forward Stepwise Selection

Forward Stepwise Selection (FSS) is a wrapper/filter hybrid method that builds the feature subset incrementally. Starting from an empty set, it greedily adds one feature at a time — the one that most improves a chosen evaluation criterion (e.g., AIC, adjusted R^2 , or accuracy) — until no further improvement is possible.

Initialization

Starts with an empty feature set $S = \{\}$. Defines the evaluation criterion (e.g., AIC for GLMs, adjusted R^2 for regression).

Subsequent Iterations

Repeats: for each remaining feature, evaluates the gain of adding it to the current set S . Adds the best one.

Advantages

- Simple and interpretable
- Computationally feasible for moderate p
- Compatible with any model that provides a criterion score

Next, the practical implementation of Stepwise Search using the daltoolbox library in R.

First Iteration

Evaluates each candidate feature individually. Adds the feature $f^* = \operatorname{argmax}_{\text{criterion}(S \cup \{f_i\})}$ to S .

Stopping Criterion

Stops when no remaining feature improves the criterion, or when a maximum number of features is reached.

Limitations

- Greedy — may miss globally optimal subsets
- Does not reconsider previously added features
- Sensitive to the choice of evaluation criterion

Reference: Hastie, T., Tibshirani, R. & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer. — Section 3.3 covers subset selection methods including forward stepwise.

Stepwise Search

Stepwise Search abandons individual ranking and instead builds a GLM model iteratively — adding or removing predictors guided by the AIC criterion.

```
library(daltoolbox)

iris <- datasets::iris
iris$IsVersicolor <- factor(
  ifelse(iris$Species == "versicolor",
    "yes", "no")
)

fs <- feature_selection_stepwise(
  "IsVersicolor",
  direction = "forward",
  family = stats::binomial
)
fs <- fit(fs, iris)
```

```
print(fs$selected)
## [1] "Species"

print(fs$ranking)
## feature score
## 1 Species    1

iris_fs <- transform(fs, iris)
head(iris_fs)
## IsVersicolor Species
```

What it is

Adds or removes predictors from a GLM iteratively, guided by an information criterion (AIC/BIC).

Binary target

Created with `ifelse`: "yes" for versicolor, "no" for all others.

Convergence warnings

Normal when a feature perfectly separates the classes (complete separation).

Result

`Species` was the only feature selected — a perfect predictor of the binary target.

 **Key concept:** Forward search starts from an empty model and adds the feature that most improves the criterion. Ref.: Hastie, Tibshirani & Friedman (2009).

Lasso, presented next, also works with regression models but uses L1 penalization to directly zero out irrelevant coefficients.

Least Absolute Shrinkage and Selection Operator (LASSO)

LASSO is a regularization method that fits a linear model while forcing irrelevant feature coefficients to shrink exactly to zero via an L_1 penalty.

$$\hat{\beta}^{lasso} = \arg \min_{\beta} \left\{ \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}$$

Intuition of L_1 Penalization

The L_1 norm creates a diamond-shaped constraint region in coefficient space. The solution occurs at the corners, where some coefficients are exactly zero.

Key Properties

Convex but non-differentiable at $\beta_j = 0$; solved via Coordinate Descent. Features must be standardized; larger $\lambda \rightarrow$ more sparsity.

Choosing λ

Typically chosen via cross-validation (`cv.glmnet`). $\lambda_{\min}$ minimizes CV error; λ_{1se} gives the most regularized model within 1 SE.

Next: CFS evaluates entire feature subsets using a correlation-based merit function.

Lasso

Lasso applies L1 penalization to linear regression — instead of iteratively selecting features, it shrinks irrelevant coefficients exactly to zero during model fitting.

```
library(daltoolbox)

iris <- datasets::iris

if (requireNamespace("glmnet",
  quietly = TRUE)) {
  fs <- feature_selection_lasso(
    "Sepal.Length"
  )
  fs <- fit(fs, iris)

  print(fs$selected)
  ## [1] "Sepal.Width" "Petal.Length"
  ##   "Petal.Width"
```

```
print(fs$ranking)
##      feature  score
## Petal.Length 0.6967400
## Sepal.Width  0.6453987
## Petal.Width  0.5291831

iris_fs <- transform(fs, iris)
head(iris_fs)
## Sepal.Length Sepal.Width
## Petal.Length Petal.Width
}
```

What it is

Regression with L1 penalty that shrinks less relevant coefficients exactly to zero.

Dependency

Requires the `glmnet` package; checked with `requireNamespace()`.

Result

3 features selected: `Petal.Length` (0.697), `Sepal.Width` (0.645), `Petal.Width` (0.529).

Continuous target

Unlike the previous examples, the target here is numeric (`Sepal.Length`).

 **Key concept:** The L1 penalty enforces sparsity — irrelevant coefficients become zero, automatically selecting features. Ref.: Tibshirani (1996).

Finally, FSS combines the incremental logic of Stepwise with exhaustive subset search, maximizing the adjusted R^2 .

Correlation-based Feature Selection (CFS)

CFS is a filter method that evaluates entire feature subsets — not individual features — using a merit function that balances two competing criteria: features should be highly correlated with the target class, but minimally correlated with each other (low redundancy).

High Correlation with Class

Each feature in the subset should have a strong correlation with the target variable — ensuring individual predictive power.

Low Correlation Among Features

Features in the subset should be weakly correlated with each other — avoiding redundancy and multicollinearity.

Merit Function

$\text{Merit}(S) = k \cdot \bar{r}_{cf} / \sqrt{(k + k(k-1) \cdot \bar{r}_{ff})}$, where k = subset size, $\bar{r}_{cf}$ = mean feature-class correlation, $\bar{r}_{ff}$ = mean feature-feature correlation.

Selection Process

1. Compute feature-class correlations ($\bar{r}_{cf}$) for all features
2. Compute feature-feature correlations ($\bar{r}_{ff}$) for all pairs
3. Search subsets (e.g., Best First, Greedy) and evaluate $\text{Merit}(S)$
4. Return the subset S^* with the highest merit score

Characteristics

- Filter method: independent of the learning algorithm
- Naturally penalizes redundant features
- Computationally efficient for moderate feature sets
- Works with both categorical and numerical data
- Sensitive to the search strategy used (Best First recommended)

 **Reference:** Hall, M.A. (1999). *Correlation-based Feature Selection for Machine Learning*. PhD Thesis, University of Waikato. — The foundational work defining the CFS merit function and its application to feature subset evaluation.

With the five methods covered theoretically, the final practical example demonstrates FSS — Forward Stepwise Subset Search — using the `leaps` package in R.

Forward Stepwise Subset Search (FSS)

FSS closes the cycle of practical methods: it uses exhaustive subset search via the `leaps` package, selecting the set of features that maximizes the adjusted R^2 of the regression model.

```
library(daltoolbox)

iris <- datasets::iris

if (requireNamespace("leaps",
  quietly = TRUE)) {
  fs <- feature_selection_fss(
    "Sepal.Length"
  )
  fs <- fit(fs, iris)

  print(fs$selected)
  ## [1] "Sepal.Width" "Petal.Length"
  ##   "Petal.Width"
```

```
print(fs$ranking)
##      feature score
## 1 Sepal.Width    1
## 2 Petal.Length    2
## 3 Petal.Width     3

iris_fs <- transform(fs, iris)
head(iris_fs)
## Sepal.Length Sepal.Width
## Petal.Length Petal.Width
}
```

What it is

Starts with no predictors and iteratively adds the feature that most improves the adjusted R^2 of the regression model.

Dependency

Requires the `leaps` package for exhaustive subset search.

Ordinal ranking

Score indicates entry order: Sepal.Width (1st), Petal.Length (2nd), Petal.Width (3rd).

Same result as Lasso

Both methods selected the same 3 features, validating the choice.

📌 **Key concept:** FSS retains the subset with the highest adjusted R^2 — a criterion that penalizes unnecessary complexity. Ref.: Hastie, Tibshirani & Friedman (2009).

With the five methods demonstrated, the deck offers a complete overview — from theory to implementation — for feature selection in R.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

